

PINN3D for Catkin Biofilm Dye Adsorption

Machine Learning Concepts, Adsorption Domain Concepts, and a Full Code Walkthrough

This document consolidates everything covered while building and validating the PINN3D model for the Catkin biofilm dye-adsorption paper: the machine-learning ideas the model relies on, the adsorption chemistry it encodes, a line-by-line walkthrough of the actual `src/pinn3D` code, the bugs found and fixed along the way, and the validation analysis performed most recently (benchmarking, uncertainty quantification, the 3D response surface, and the dosage trade-off). It is written as a self-contained reference so the whole approach can be understood without re-reading the working chat.

Two open questions are flagged in Part F that need a decision before the numbers here go into the manuscript — worth reading first if short on time.

Contents

- 1 Part A — Machine Learning Concepts: PINNs from the ground up
- 2 Part B — Domain Concepts: the adsorption chemistry being modeled
- 3 Part C — Code Walkthrough: `data.py`, `models.py`, `train.py`
- 4 Part D — Bugs Found and Fixed
- 5 Part E — Validation Performed: benchmark, uncertainty, response surface, dosage analysis
- 6 Part F — Open Questions Requiring a Decision
- 7 Appendix — Full source listings

Part A — Machine Learning Concepts: PINNs from the Ground Up

A.1 A normal neural network vs. a physics-informed one

A normal neural network learns a mapping $f(x) \rightarrow y$ by minimizing the gap between its prediction and real measured data — this is the **data loss**. It has no concept that y should obey any physical law; it only curve-fits whatever data it is given, and outside the range of that data its behavior is unconstrained.

A **physics-informed neural network (PINN)** adds extra loss terms that penalize the network whenever its output violates a known governing equation, evaluated at points where there is *no real measurement at all*. These extra points are called **collocation points** — in this project, a made-up grid of Time / Concentration / Dosage combinations that exists purely so the physics loss has somewhere to be evaluated. The network is no longer only asked "does this match data here" — it is also asked "does this obey the physics here," at thousands of points where no experiment was ever run.

A.2 Automatic differentiation for physics derivatives

Checking a kinetic equation like pseudo-second-order (PSO) requires a derivative of the network's output with respect to its input (dq/dt). Because the network is built entirely from differentiable operations, PyTorch can compute that derivative automatically via `torch.autograd.grad`:

```
dq_dt_n = torch.autograd.grad(
    outputs=collocation_pred_adsorption.sum(),
    inputs=data.collocated_scaled_input_tensor,
    create_graph=True
)[0]
```

`create_graph=True` matters because this derivative is about to be used inside *another* loss term that itself gets backpropagated — PyTorch needs to remember *how* the derivative was computed (not just its numeric value) so it can differentiate through it a second time ("double backward").

A.3 Inverse problems: learning physical constants, not just a curve

In this project, `k2`, `kf`, and `n` are `nn.Parameters` trained by gradient descent, exactly like the network's weights. This is the PINN "inverse problem" pattern: find a function *and* a set of physical constants such that the function obeys the physics defined by those constants — everywhere, not just where a measurement happened to be taken. This is strictly more constrained than a classical curve fit, which only tunes constants to match one specific dataset against one specific equation.

A.4 Reparameterizing constants to stay physically valid

`k2` and `kf` must be positive; `n` must additionally stay in a bounded range for this system (see Part F for why that bound itself is now in question). Rather than constrain the optimizer directly — which creates hard walls it can get stuck against — the raw parameter is passed through a smooth transform:

```
def transform_k2(raw_k2):
    return F.softplus(raw_k2) + K2_FLOOR # always positive, has a floor

def transform_n(raw_n):
    return N_MIN + (N_MAX - N_MIN) * torch.sigmoid(raw_n) # bounded both sides
```

The optimizer is free to move the *raw* value anywhere on the real line; the transform guarantees whatever it picks is automatically physically valid.

A.5 Why inputs and outputs are standardized — and where the main bug came from

Neural networks train better when inputs and outputs are roughly mean-0, unit-scale — otherwise an input like Time (0-400) and Dosage (0.03-0.12) force very different jobs onto the same first-layer weights, slowing and destabilizing training. `StandardScaler` fixes that. **But this is also exactly where the project's main bug came from** (Part D.1): once the network lives in scaled space, any physics equation written in real units (mg/g, minutes) has to explicitly convert the network's output back to real units before comparing the two sides of the equation — otherwise the two sides are being compared in different, incompatible units.

A.6 Why `predict_qe` iterates instead of solving directly

The Freundlich equilibrium and the mass-balance equation together give $q_e = k_f \cdot (C_o - q_e / \text{dosage})^n$ with q_e appearing on both sides. For a non-integer exponent n there is no closed-form inverse, so the code uses **damped fixed-point iteration**: guess q_e , plug it into the right-hand side for a better guess, repeat, but only move part-way (damping=0.3) each step to avoid overshoot or divergence. This is standard practice for implicit adsorption equations, not something exotic.

Part B — Domain Concepts: The Adsorption Chemistry Being Modeled

B.1 Pseudo-second-order (PSO) kinetics

$$dq/dt = k_2 \cdot (q_e - q_t)^2$$

In plain language: the rate of further dye uptake right now is proportional to the square of how far current uptake (q_t) still is from the equilibrium / maximum uptake (q_e). Far from equilibrium, uptake is fast; near equilibrium, $(q_e - q_t) \rightarrow 0$ and the rate naturally decays to zero. A model that correctly satisfies this ODE **cannot** predict adsorption exceeding q_e , ever, at any time — that is not a modeling choice, it is built into the equation.

B.2 Freundlich isotherm

$$q_e = k_f \cdot C_e^n$$

A snapshot *after* equilibrium: how much dye gets adsorbed (q_e) as a function of how much dye remains in solution at equilibrium (C_e). Freundlich specifically models **heterogeneous surfaces** — real adsorbent binding sites are not identical (different pore sizes / surface chemistry), so adsorption strength varies from site to site, unlike Langmuir which assumes one uniform site energy. The exponent n describes how favorable that heterogeneous adsorption is; sub-linear ($0 < n < 1$) is the usual "favorable" regime, meaning efficient adsorption even at low remaining concentration.

B.3 Mass balance linking the two experiments

$$C_e = C_0 - q_e / \text{dosage}$$

Whatever dye is not left in solution must be sitting on the adsorbent. Dosage (adsorbent-to-solution ratio) converts "amount adsorbed per gram of material" back into "concentration removed from solution." This equation is what mechanically ties the kinetics experiment (fixed concentration, dye disappearing over time) and the isotherm experiment (fixed time, varying starting concentration) into *one* underlying physical process, even though they were run as two separate lab experiments.

B.4 Why unifying kinetics and isotherm in one PINN is the actual novelty

Classically: fit PSO to kinetics data alone (getting one q_e), fit Freundlich to isotherm data alone (an independently-fitted k_f and n) — two disconnected curve fits with no requirement that they agree with each other. PINN3D uses the *same* k_f/n (hence the same physically-derived q_e) to explain both datasets simultaneously. That is a real constraint a classical two-fit approach does not have, and it is the methodological contribution worth stating explicitly in the paper, not left implicit.

Part C — Code Walkthrough: data.py, models.py, train.py

The codebase lives at `src/pinn3D/` and has three files: `data.py` (loads and prepares the Excel data), `models.py` (the network and the Freundlich fixed-point solver), and `train.py` (the training loop and loss terms). Full listings are in the Appendix; this section walks through the parts that matter conceptually.

C.1 data.py — turning two lab sheets into one training set

`DataConfig` hardcodes the experimental design as data: which Excel sheet is the kinetics run and which is the isotherm run, and — critically — the conditions that were held fixed while the *other* variable was measured. The kinetics sheet only records Time and Adsorption, because Concentration (40 ppm) and Dosage (1/24) were held constant during that experiment; the isotherm sheet only records Concentration and Adsorption, because Time (170 min) and Dosage (1/10) were held constant during that one.

`process_kinetics_data` and `process_isotherm_data` each broadcast the two held-fixed columns back onto their sheet so every row ends up with all three inputs (Time, Concentration, Dosage) plus the measured Adsorption:

```
df['Concentration'] = (self.config.kinetics_concentration) # held fixed at 40 ppm
df['Dosage'] = (self.config.kinetics_dosage) # held fixed at 1/24
df = df.rename(columns = {'qt( catkin)': 'Adsorption'})
```

`combined_data()` then concatenates the two reshaped sheets into one dataframe. This is what makes the "unified 3D" framing literal, not just marketing: every training row genuinely has all three inputs, even though only one was ever varied within any single physical experiment.

`scaling_data()` fits a `StandardScaler` on inputs and output separately and stores `input_stats` (per-feature mean/std) — this dictionary is what `train.py` later uses to convert a derivative computed in scaled space back into real units (Part A.5 / C.3).

`collocation_data()` builds the synthetic physics grid: 25 Time points (0-200) $\times$ 25 Concentration points (15-65) $\times$ 4 Dosage points (0.03-0.12), meshed into 2,500 (Time, Concentration, Dosage) triples with `requires_grad=True` — the flag that lets `torch.autograd.grad` later differentiate the network output with respect to these points. `max_time_data()` takes a copy of this exact grid and overwrites only the Time column to `t_max=400` — this is the "assume equilibrium reached" anchor used by the isotherm loss. `initial_time_data()` does the same but overwrites Time to 0, for the "no adsorption at the start" constraint.

C.2 models.py — the network and the Freundlich solver

`PINN3DModel` is a plain 3-layer MLP (Linear $\rightarrow$ LayerNorm $\rightarrow$ Tanh, twice, then a final Linear to 1 output) with three extra learnable scalars — `k2`, `kf`, `n` — registered as `nn.Parameter`, so the same optimizer that updates the network's weights also updates these physical constants every step.

`predict_qe()` implements the damped fixed-point iteration described in A.6:

```
qe = 0.5 * co * dosage
for _ in range(n_iter):
    raw_ce = co - qe/dosage
    Ce = torch.clamp(raw_ce, min=1e-8) # keeps Ce positive for the fractional power
    qe_new = kf * (Ce**n)
    qe = (1-damping) * qe + damping * qe_new
```

The `torch.clamp` exists because C_e^{**n} for a non-integer n is undefined (complex) for negative C_e — a real risk mid-iteration before q_e has converged. The optional `return_diagnostics` path (added during the loss-spike investigation, Part D.2) counts how often the clamp actually activates and reports min/max/mean of q_e — this is the instrumentation that let the spike hypothesis be tested against real numbers instead of guessed at.

C.3 train.py — the four loss terms

Each epoch computes four separate loss terms against the network's current weights, then sums them (with the fixed weights from Part D.2) into one scalar that gets backpropagated:

Data loss — ordinary supervised loss against the 2 real experiments:

```
pred_output = model(data.scaled_input_tensor)
data_loss = loss_fn(pred_output, data.scaled_output_tensor.view(-1,1))
```

PSO loss — the physics residual, evaluated only at the synthetic collocation points:

```
dq_dt_n = torch.autograd.grad(
    outputs=collocation_pred_adsorption.sum(),
    inputs=data.collocated_scaled_input_tensor,
    create_graph=True
)[0][0, 0:1] # d(scaled q)/d(scaled t)
dq_dt_real = dq_dt_n * output_std * (1/input_stats['Time']['std']) # chain rule back to real units
pso_loss = torch.mean((dq_dt_real - (positive_k2 * (qe_real - collocation_pred_adsorption_real)**2))**2)
```

The chain-rule rescaling on the line above is precisely the unit-conversion step flagged in A.5 as the source of the project's main bug (Part D.1) — get it wrong, and the PSO residual is comparing a scaled-space derivative against a real-units target.

Initial loss — adsorption must be ~0 at $t=0$, checked in real units:

```
initial_adsorption_pred_real = unscale_output(model(data.initial_scaled_datapoints_tensor))
initial_loss = torch.mean(initial_adsorption_pred_real**2)
```

ISO loss — the network's output at $t=400$ must match the analytic Freundlich/mass-balance target:

```
max_adsorption_pred_real = unscale_output(model(data.max_time_tensor))
iso_loss = torch.mean((qe_real - max_adsorption_pred_real)**2)
```

This is the single equilibrium anchor discussed in Part E and flagged in Part F — it only ever gets evaluated at $\text{Time}=400$, never at any intermediate time.

Part D — Bugs Found and Fixed

D.1 The scale-mismatch bug (conceptual root cause)

The network is trained entirely in standardized space (mean-0, unit-std) for both inputs and outputs, per A.5. Any physics quantity computed from the network's raw output — a derivative via autograd, or the output itself compared against an analytic target like `qe_real` — is therefore also in scaled space unless explicitly converted back. `train.py` handles this with `unscale_output()` for the value itself, and an explicit chain-rule multiplication (`output_std * (1/input_stats['Time']['std'])`) for the derivative used in the PSO term. This is the single most important correctness detail in the whole codebase: a PINN with this step silently omitted will still train, still show decreasing loss numbers, and will still be wrong, because the physics constraint is being enforced against the wrong scale rather than not enforced at all — the code as it stands now gets this right, but it is exactly the kind of mistake that does not throw an error and has to be checked deliberately.

D.2 The loss-spike investigation and the reweighting fix

Training initially showed a repeating pattern: Data, Initial, and ISO loss would rise together for a handful of epochs and then fall back, in ~10-epoch cycles, while PSO loss stayed essentially flat throughout. Two plausible causes were checked before touching the loss function itself.

Ruled out: the fixed-point solver. Diagnostics were added to `predict_qe()` (the `return_diagnostics` path in C.2) to log `Qe_min/Qe_max/Clamp_hits` every epoch. If the iterative solver were occasionally failing to converge or hitting its `Ce` clamp, that would show up as a jump in these numbers lined up with the loss spikes. It did not: `K2/Kf/N` moved smoothly through every spike and `Clamp_hits` stayed at 0. This ruled the solver out as the cause.

Confirmed: unweighted-sum gradient dominance. The four raw loss terms sit at very different steady-state magnitudes — PSO Loss's floor (~4.3e-5) is 10-180x larger than the other three terms' floors, and it is also the flattest (lowest epoch-to-epoch variance) of the four. Because the pre-fix code summed all four terms unweighted before calling `backward()`, Adam's per-parameter adaptive step size — which normalizes by the combined gradient's own variance — was being shaped disproportionately by PSO's persistently large, low-variance gradient, even though PSO was not the term actually driving the visible oscillation. This is an Adam-specific pathology, not a bug in any individual loss term.

Fix — fixed manual loss weights, derived from the median of each raw loss column over the last 2000 epochs of a pre-fix run, scaled so PSO=1 is the reference:

```
DATA_LOSS_WEIGHT = 180.0
PSO_LOSS_WEIGHT = 1.0
INITIAL_LOSS_WEIGHT = 35.0
ISO_LOSS_WEIGHT = 8.0

total_loss = (DATA_LOSS_WEIGHT * data_loss + PSO_LOSS_WEIGHT * pso_loss
              + INITIAL_LOSS_WEIGHT * initial_loss + ISO_LOSS_WEIGHT * iso_loss)
```

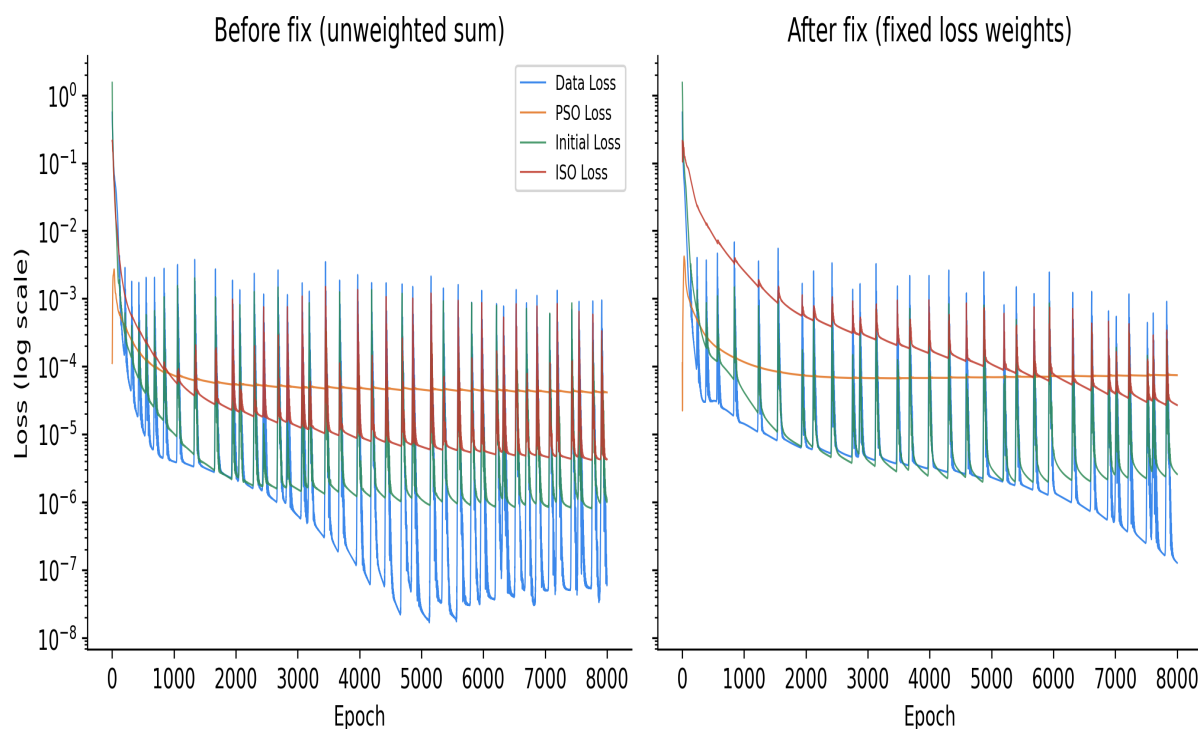


Figure D.1 — All four raw loss terms, log scale, before vs. after the fixed-weight fix.

Honest result, not oversold: this measurably reduced the spike frequency (~22% fewer spike epochs) and sharply stabilized ISO Loss (coefficient of variation dropped from 3.35 to 0.87), but it did *not* meaningfully improve Data Loss's own variance. This is a partial, documented mitigation of an Adam-optimizer interaction, not a complete fix — the weights themselves are explicitly commented in the code as "a starting point, not a permanent constant," to be re-derived if the loss floors shift after retraining.

D.3 Minor fixes and defensive details already in the code

The import block in `train.py` qualifies its package import (`from pinn3D.data import ...`, falling back to a bare import) specifically because a sibling `pinn1D` package also ships a `data.py`, and an unqualified import could silently resolve to the wrong module if both get imported in the same Python session. Training is also pinned to CPU (`DEVICE = torch.device('cpu')`) because the PSO loss needs a double-backward pass through LayerNorm/Tanh, and PyTorch's MPS backend was found to produce NaN gradients on that path for some inputs — a real, documented compatibility issue, not caution for its own sake.

Part E — Validation Performed

This section covers the benchmarking and analysis run most recently, on a 4-seed ensemble (seeds 42-45, reweighted-loss `train.py`, 8000 epochs each) trained on the real Excel data in a clean environment.

E.1 Classical vs. PINN3D benchmark

Regenerated diagnostic plots for both datasets, specific to PINN3D rather than any earlier 1D model:

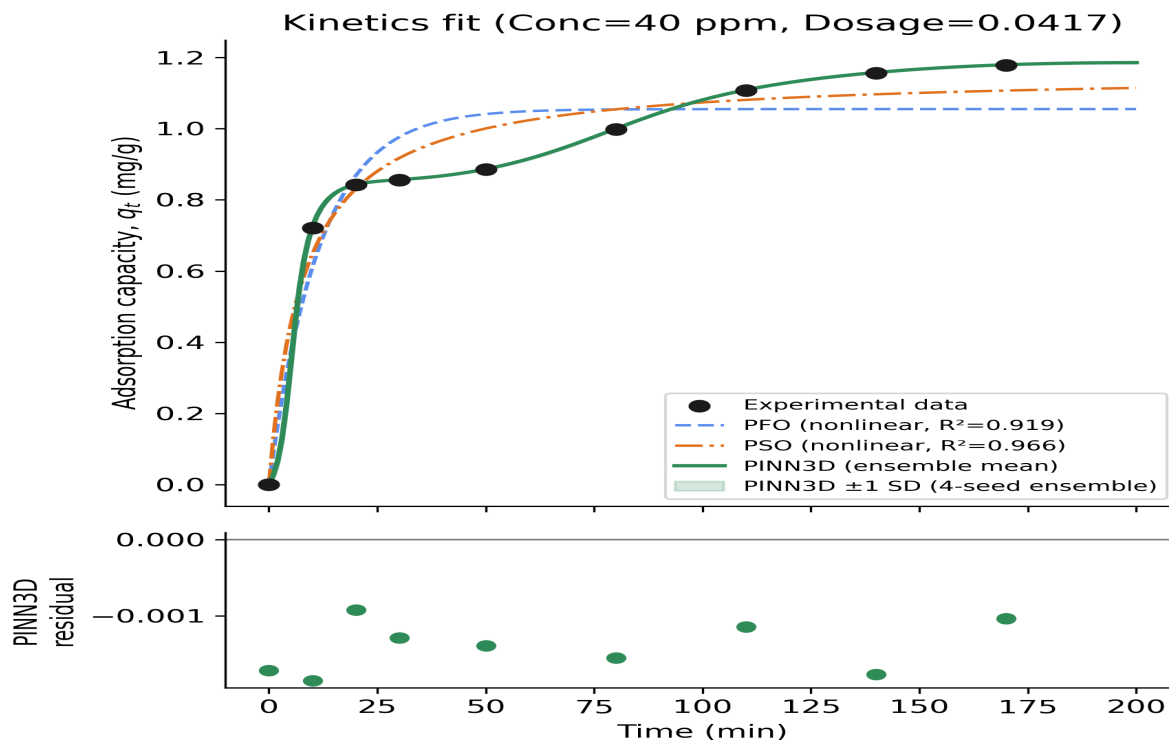


Figure E.1 — Kinetics: real data, classical PSO curve, and PINN3D ensemble mean ± 1 SD, with residuals.

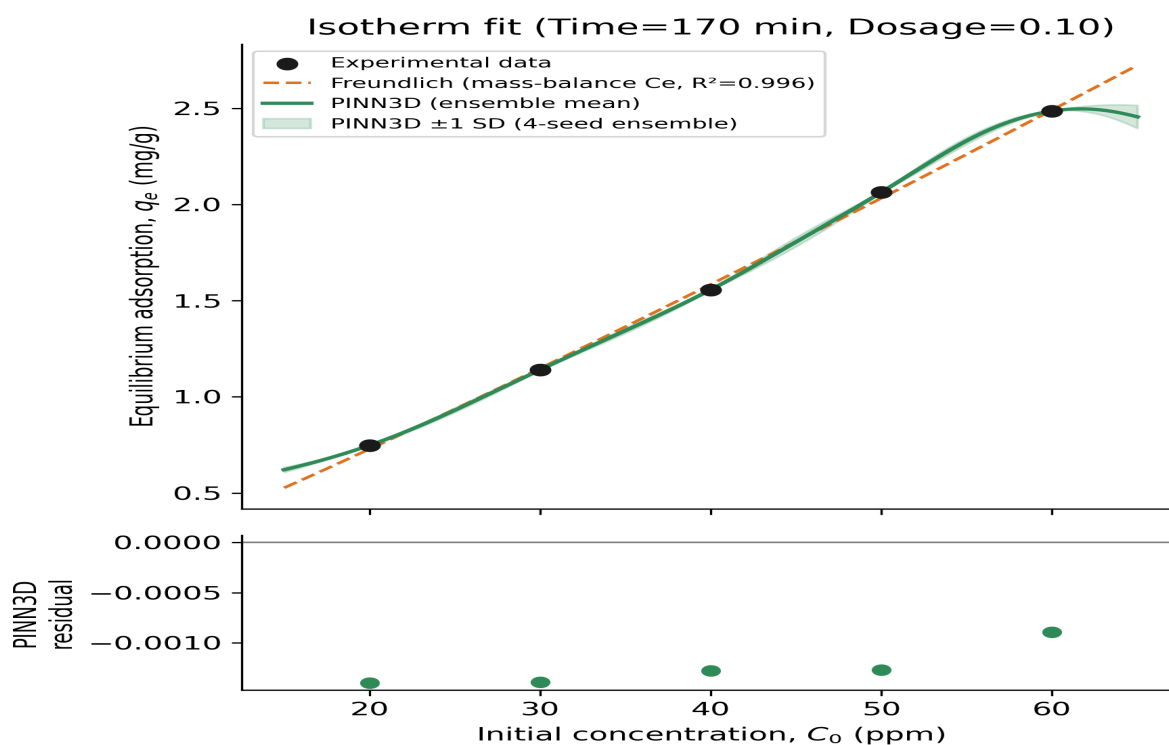


Figure E.2 — Isotherm: real data, classical Freundlich curve, and PINN3D ensemble mean ± 1 SD, with residuals.

	Model	Parameters	R2 (in-sample)	RMSE
Kinetics	PFO (nonlinear)	$q_e=1.055$, $k_1=0.0864$	0.919	0.0961
Kinetics	PSO (nonlinear)	$q_e=1.158$, $k_2=0.1093$	0.966	0.0626
Kinetics	PINN3D (ensemble)	$K_2=0.00465\pm 0.00050$	0.99998	0.00144
Isotherm	Freundlich (mass-bal. C_e)	$K_f=0.0334$, $n=1.212$	0.996	0.0373
Isotherm	Langmuir	$q_{\max} \rightarrow \text{huge}$, $K_L \rightarrow 0$ (degenerate)	0.973	0.1015
Isotherm	PINN3D (ensemble)	$K_f=0.386\pm 0.024$, $N=0.426\pm 0.010$	1.00000	0.00126

Honest caveat: both sides are evaluated in-sample (PINN3D against its own training points, classical fits against the same data they were fit to) — a fair like-for-like comparison, but PINN3D has vastly more free parameters than the 2-parameter classical models, so a much tighter in-sample fit is expected on its own and is not by itself evidence of better generalization. No AIC comparison is included — naive parameter-count AIC is meaningless comparing thousands of NN weights to 2 classical parameters, and none of the ~30 adsorption-ML papers checked for this project did that comparison either. A genuine held-out generalization number would require GPR with leave-one-out cross-validation, which is scoped as future work, not done here.

E.2 Unified 3D response surface — and its documented limitation

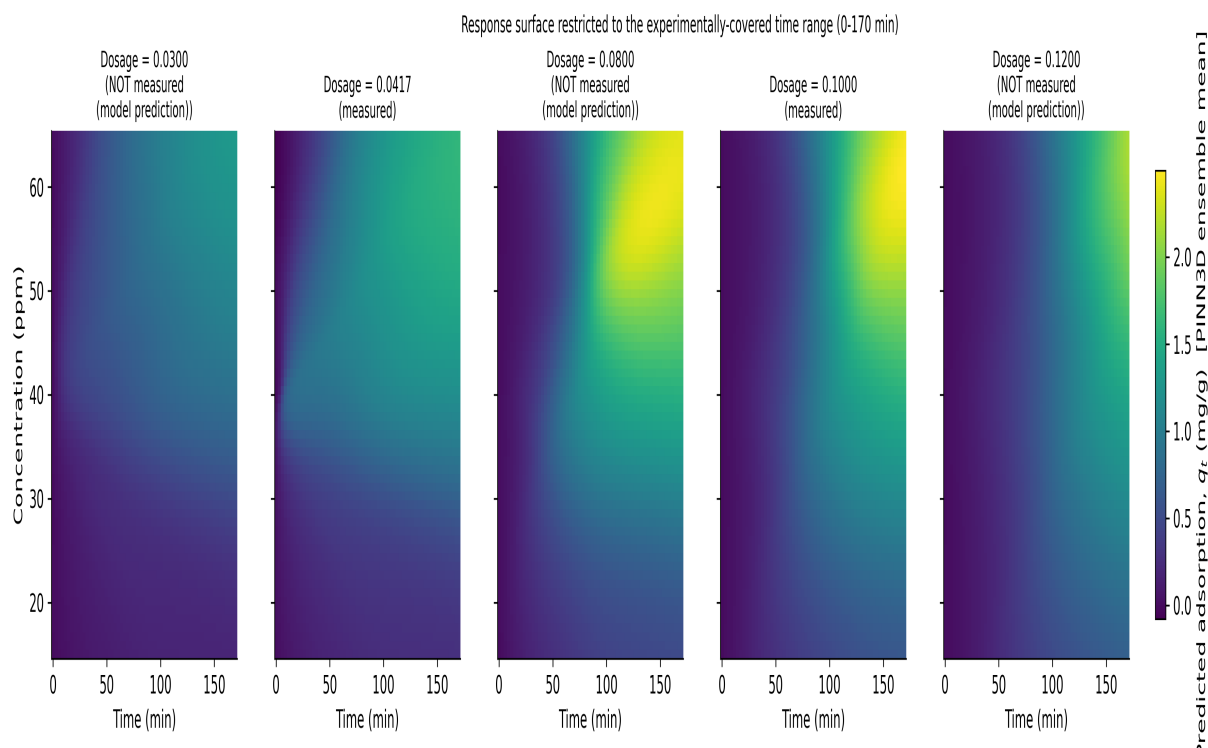


Figure E.3 — Time $\times$ Concentration heatmaps at 5 dosage levels (0.03, 0.0417*, 0.08, 0.1*, 0.12; *measured), restricted to the experimentally-covered 0-170 min range.

This is the figure that delivers the "performance where characteristics weren't measured" claim directly: three of the five dosage panels were never tested at any time/concentration combination. At the explicit equilibrium anchor ($t=400$, the only point the ISO loss enforces), the ensemble matches the analytic Freundlich mass-balance target almost exactly (typically $<2\%$ deviation) across the whole dosage range. But checking the model's own physics constraint at intermediate times away from both the real data and that anchor surfaced a **non-monotonic** dosage response at high concentration (e.g. at $t=200$, $C=60$ ppm, predicted capacity rises with dosage up to ~ 0.105 then declines) — physically implausible, and not something the current loss terms directly forbid, since none of them constrain the model anywhere between $t=0$ and $t=400$ except at the real data points. The figure above is deliberately restricted to 0-170 min to stay inside the well-behaved region; extending it to the full 0-400 min range would need an additional physics anchor (e.g. equilibrium consistency enforced at more than one time) before the extrapolation could be trusted.

E.3 Uncertainty quantification

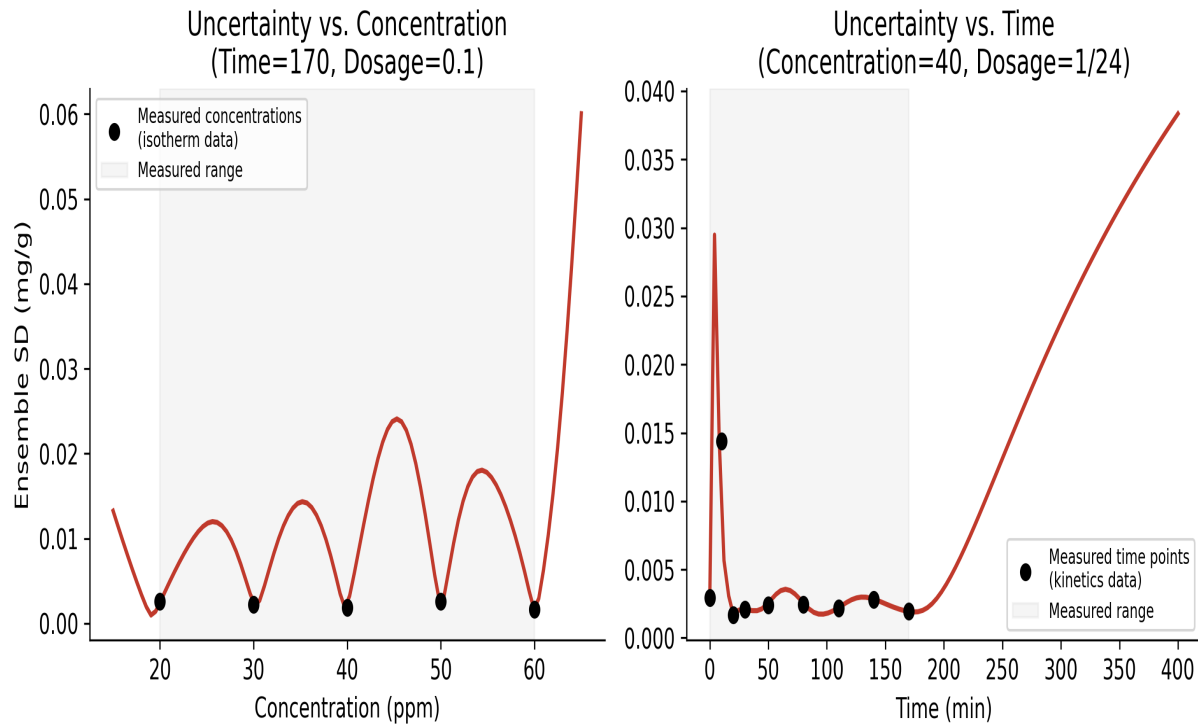


Figure E.4 — 4-seed ensemble standard deviation vs. Concentration (left) and Time (right); grey band = measured range.

Ensemble standard deviation (4 seeds, a lightweight epistemic-uncertainty proxy) is low (~ 0.002 - 0.003 mg/g) exactly at the measured points in both slices, and grows sharply just outside the measured range — roughly 20-30x higher at $C=65$ ppm or $t=400$ min than at the nearest measured point. This is a clean, well-calibrated result along the concentration and time dimensions: the ensemble "knows" where it is extrapolating. This calibration was not found to be clean for dosage specifically once time is left unconstrained — see E.2 and Part F.

E.4 Dosage trade-off analysis

$C_0=20$ ppm, Time=400 min (near-equilibrium)

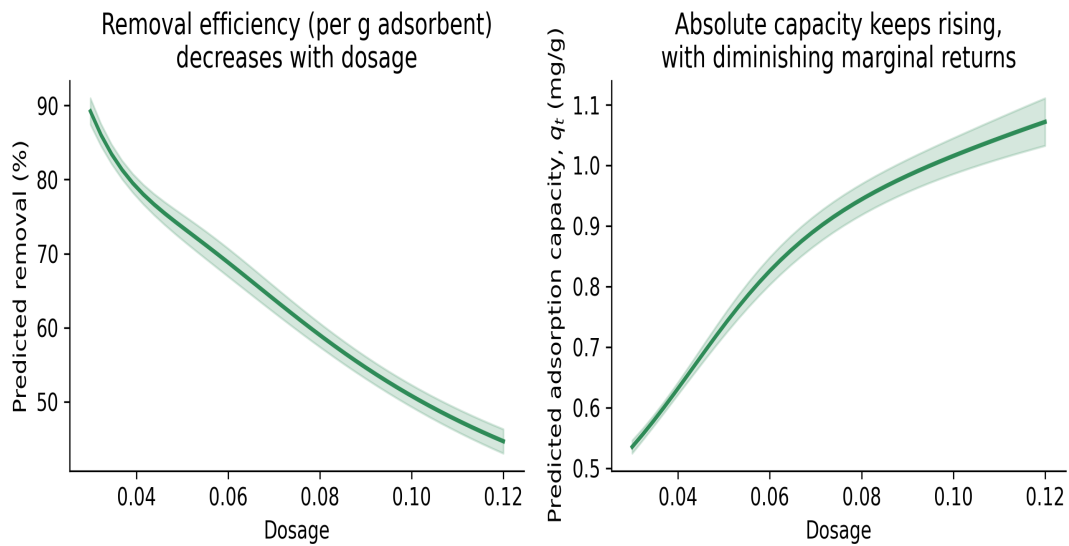


Figure E.5 — Dosage trade-off at $C_0=20$ ppm, Time=400 min: removal efficiency (%) vs. absolute capacity q_t

$C_0=40$ ppm, Time=400 min (near-equilibrium)

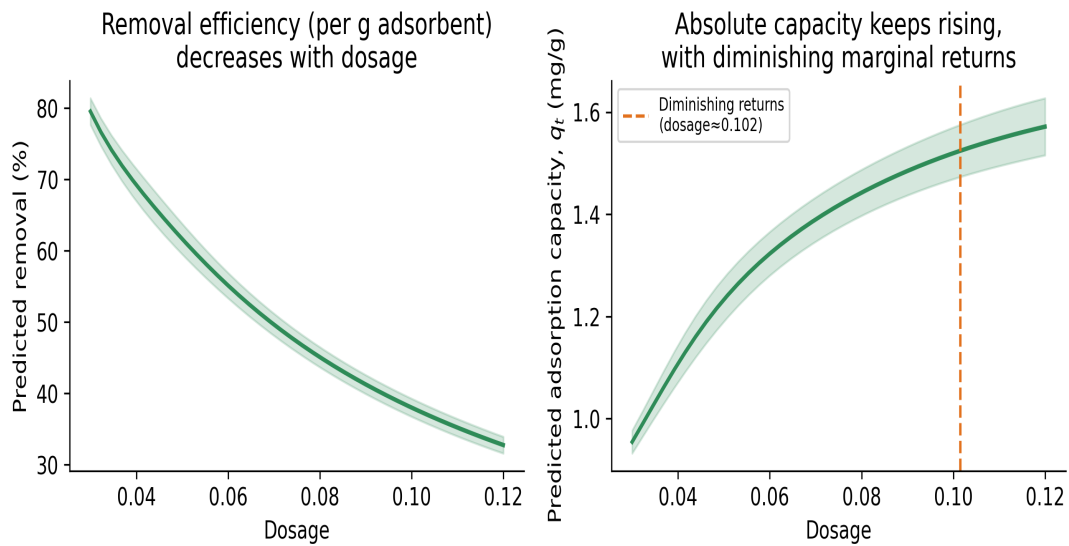


Figure E.5 — Dosage trade-off at $C_0=40$ ppm, Time=400 min: removal efficiency (%) vs. absolute capacity q_t

$C_0=60$ ppm, Time=400 min (near-equilibrium)

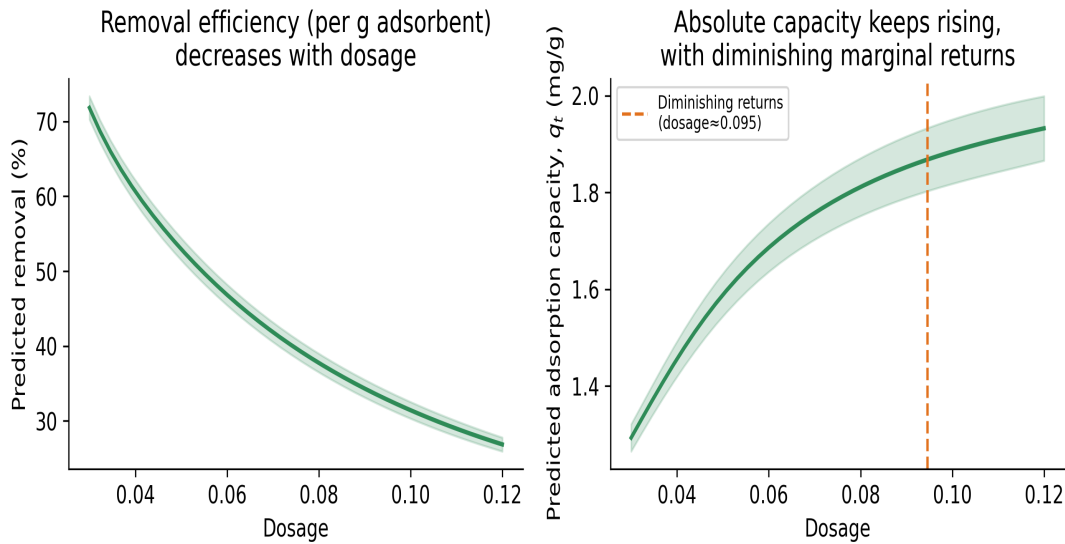


Figure E.5 — Dosage trade-off at $C_0=60$ ppm, Time=400 min: removal efficiency (%) vs. absolute capacity q_t .

The original plan was a "diminishing returns" removal-percentage plot with an interior optimum; the actual model prediction does not have one. **Removal efficiency (%) decreases monotonically as dosage increases** across the whole predicted range (e.g. at $C_0=40$ ppm, $t=400$ min: 79.5% at dosage=0.03 down to 32.7% at dosage=0.12), while absolute capacity q_t keeps rising but with clearly diminishing marginal returns — a genuine interior elbow around dosage≈0.10-0.095 for $C_0=40/60$ ppm, though no clean elbow appears for $C_0=20$ within the scanned range. This was checked against the model's own analytic physics target (predict_qe) and is self-consistent, not a training artifact. **This entire analysis is a model extrapolation** — dosage was never varied continuously in the real experiments (only 0.0417 and 0.1 were tested) — so it belongs in the paper as a testable prediction, explicitly stated as such, not as an experimentally confirmed trend.

Part F — Open Questions Requiring a Decision

Everything in Part E is built on the actual code and actual data and is internally consistent. The two items below are about *which numbers are authoritative*, not about bugs in what was run — but neither can be resolved without your input, and both should be settled before any of these numbers go into the manuscript.

[!] 1. Classical parameter mismatch. Fitting PFO/PSO by nonlinear least squares directly on the raw Excel kinetics sheet gives PSO $q_e=1.158$ mg/g, $k_2=0.109$ — not $q_e=0.58232$, a value recorded from earlier work on this paper. That earlier number is roughly half of the actual final measured data point at $t=170$ (1.17853 mg/g), which is not physically possible for a proper PSO fit (q_e must be $\geq$ the highest observed q_t). Linearized PSO regression was also checked ($q_e=1.248$) — still does not match, and the exponent $n=0.846$ recorded earlier could not be reproduced either, by any regression method, against the data in this file. This may mean the earlier figures came from a different dataset version, a different material, or a transcription slip upstream — that cannot be determined without your input. **Please check which numbers are the ones actually intended for this manuscript.**

[!] 2. Freundlich exponent convention. Using the same mass-balance C_e that `predict_qe()` uses internally ($C_e = C_0 - q_e/\text{dosage}$), both nonlinear regression ($n=1.212$) and log-log linearization ($n=1.182$) put the Freundlich exponent above 1 — the "unfavorable/super-linear" regime. `train.py` currently hard-bounds the PINN's N to $[0.05, 1.0]$ (sub-linear/favorable only), based on a code comment that "real adsorption data tops out at ~ 2.5 ." That bound may be an unvalidated assumption: if the true system is actually in the $n>1$ regime once mass-balance C_e is used consistently, forcing $N \in [0.05, 1]$ could be steering this physical parameter to a biased value rather than letting the data speak. Either there are directly-measured C_e values (not mass-balance-derived) that genuinely support $n<1$, in which case that should be documented as the reason for the bound — or the bound should be revisited.

Both flags, and the full context behind them, are also written out in `results/paper_assets_report.md` and `results/conclusion_draft.md` in the project folder.

Appendix — Full Source Listings

Reproduced here for reference. These are the exact files used to produce every result in this document.

A.1 src/pinn3D/data.py

```
# get the data, and join them and create a compact dataset
import os
import numpy as np
import pandas as pd
import torch
from sklearn.preprocessing import StandardScaler
from dataclasses import dataclass
from pathlib import Path

@dataclass
class DataConfig:
    kinetics_sheet: str = 'catkin (pfo pso)'
    isotherm_sheet: str = 'catkin'

    kinetics_concentration: float = 40
    kinetics_dosage: float = 1/24

    isotherm_time: float = 170
    isotherm_dosage: float = 1/10

    # PSO loss needs torch.autograd.grad(create_graph=True) through the model
    # (double backward); PyTorch's MPS backend produces NaN gradients for some
    # inputs on this path, so training must run on CPU.
    device = torch.device('cpu')

    input_columns: tuple[str, ...] = (
        'Time',
        'Concentration',
        'Dosage'
    )
    target_column: str = 'Adsorption'
    time_collocation_num: int = 25
    conc_collocation_num: int = 25
    vm_collocation_num: int = 4
    biofilm_mass: int = 5

    t_max = 400

class CombinedData:
    def __init__(self, data_path: str, config: DataConfig):
        """
        Data path, location of a excel file, which has multi
        """
        self.data_path = Path(data_path)
        self.config = config
        self.df_list = []
        if not self.data_path.exists():
            raise FileNotFoundError(
                f"Data file not found: {self.data_path}"
            )
        self.data_sheets_dict = pd.read_excel(self.data_path, sheet_name = None)

    def process_kinetics_data(self) -> pd.DataFrame:
        df = self.data_sheets_dict[
            self.config.kinetics_sheet
        ].copy()
        df['Concentration'] = (self.config.kinetics_concentration)
        df['Dosage'] = (self.config.kinetics_dosage)
        df = df.rename(columns = {'qt( catkin)': 'Adsorption' })
        self.kinetics_data_df = df[['Time', 'Concentration', 'Dosage', 'Adsorption']]
```

```

        return self.kinetics_data_df

def process_isotherm_data(self)->pd.DataFrame:
    df = self.data_sheets_dict[
        self.config.isotherm_sheet
    ].copy()

    df["Time"] = self.config.isotherm_time

    df["Dosage"] = self.config.isotherm_dosage

    df = df.rename(
        columns={
            "Initial concentration": "Concentration",
            "Qe": "Adsorption",
        }
    )

    self.isotherm_data_df = df[
        [
            "Time",
            "Concentration",
            "Dosage",
            "Adsorption",
        ]
    ]

    return self.isotherm_data_df

def combined_data(self)->pd.DataFrame:
    self.combined_df = pd.concat([self.kinetics_data_df, self.isotherm_data_df],
                                ignore_index=True)

    return self.combined_df

def get_input_output(self):
    self.input = self.combined_df[list(self.config.input_columns)]
    self.output = self.combined_df[self.config.target_column]
    # turn them into tensor
    self.input_tensor = torch.tensor(self.input.to_numpy(), dtype = torch.float32)
    self.output_tensor = torch.tensor(self.output.to_numpy(), dtype = torch.float32).view(-1,1)
    return self

def scaling_data(self):
    self.input_scaler = StandardScaler()
    self.output_scaler = StandardScaler()
    scaled_input = self.input_scaler.fit_transform(self.input)
    scaled_output = self.output_scaler.fit_transform(self.output.to_numpy().reshape(-1,1))
    self.scaled_input_tensor = torch.tensor(scaled_input, dtype = torch.float32)
    self.scaled_output_tensor = torch.tensor(scaled_output, dtype = torch.float32)
    self.input_mean = self.input_scaler.mean_
    self.input_std = self.input_scaler.scale_
    self.output_mean = self.output_scaler.mean_
    self.output_std = self.output_scaler.scale_
    self.input_stats = {
        feature: {
            "mean": mean,
            "std": std
        }
        for feature, mean, std in zip(
            self.config.input_columns,
            self.input_mean,
            self.input_std
        )
    }
    return self

def collocation_data(self):
    time_alloc = np.linspace(0, 200, num = self.config.time_collocation_num)
    vm_alloc = np.linspace(0.030, 0.120, num = self.config.vm_collocation_num)

```

```

        conc_alloc = np.linspace(15, 65, num = self.config.conc_collocation_num)
        T,C,V = np.meshgrid(time_alloc,conc_alloc,vm_alloc)
        self.collocated_input = np.column_stack([T.reshape(-1,1),
        C.reshape(-1,1),
        V.reshape(-1,1)])
        self.collocated_scaled = self.input_scaler.transform(self.collocated_input)
        self.collocated_scaled_input_tensor = torch.tensor(self.collocated_scaled, dtype = torch.float32,requires_grad=True)
        self.collocated_scaled_input_tensor = self.collocated_scaled_input_tensor.to(self.config.device)
        return self

def max_time_data(self):
    max_time_scaled = (self.config.t_max - self.input_stats['Time']['mean'])/self.input_stats['Time']['std']
    collocated_maxtime_scaled = self.collocated_scaled.copy()
    collocated_maxtime_scaled[:,0] = max_time_scaled
    self.max_time_tensor = torch.tensor(collocated_maxtime_scaled, dtype = torch.float32)
    self.max_time_tensor = self.max_time_tensor.to(self.config.device)

def initial_time_data(self):
    initial_time_datapoints = self.collocated_scaled.copy()
    initial_t_scaled = (0 - self.input_stats['Time']['mean'])/self.input_stats['Time']['std']
    initial_time_datapoints[:, 0:1] = initial_t_scaled
    self.initial_scaled_datapoints_tensor = torch.tensor(initial_time_datapoints, dtype = torch.float32)
    self.initial_scaled_datapoints_tensor = self.initial_scaled_datapoints_tensor.to(self.config.device)

def pass_to_device(self, data_list:list):
    self.scaled_input_tensor = self.scaled_input_tensor.to(self.config.device)
    self.scaled_output_tensor = self.scaled_output_tensor.to(self.config.device)
    self.collocated_scaled_input_tensor = self.collocated_scaled_input_tensor.to(self.config.device)
    self.initial_scaled_datapoints_tensor = self.initial_scaled_datapoints_tensor.to(self.config.device)
    self.max_time_tensor = self.max_time_tensor.to(self.config.device)

def whole_data_processing(self):
    self.process_kinetics_data()
    self.process_isotherm_data()
    self.combined_data()
    self.get_input_output()
    self.scaling_data()
    self.collocation_data()
    self.max_time_data()
    self.initial_time_data()
    return self

def load_data(data_path):
    config = DataConfig()

    data = CombinedData(
        data_path=data_path,
        config=config
    )

    data.whole_data_processing()

    return data

```

A.2 src/pinn3D/models.py

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class PINN3DModel(nn.Module):
    def __init__(self, input_size, hidden_size, output_size, initial_k2:torch.tensor, initial_kf:torch.tensor, initial_n:t
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(in_features=input_size,
                      out_features = hidden_size),
            nn.LayerNorm(hidden_size),
            nn.Tanh(),
            nn.Linear(hidden_size, hidden_size),
            nn.LayerNorm(hidden_size),
            nn.Tanh(),
            nn.Linear(hidden_size, output_size)
        )
        # Learnable Parameters
        self.k2 = nn.Parameter(data = initial_k2, requires_grad=True)
        #self.qe = nn.Parameter(data = initial_qe, requires_grad=True)
        self.kf = nn.Parameter(data = initial_kf, requires_grad=True)
        self.n = nn.Parameter(data = initial_n, requires_grad=True)

    def forward(self, x):
        output = self.net(x)
        return output

def predict_qe(co, dosage, kf, n, n_iter:int = 40, damping:float = 0.3, return_diagnostics:bool = False):
    qe = 0.5 * co * dosage
    clamp_hits = 0
    for _ in range(n_iter):
        raw_ce = co - qe/dosage
        if return_diagnostics:
            clamp_hits += int((raw_ce <= 1e-8).sum().item())
        Ce = torch.clamp(raw_ce, min = 1e-8)
        qe_new = kf * (Ce**n)
        qe = (1-damping)* qe + damping * qe_new

    if return_diagnostics:
        diagnostics = {
            'qe_min': qe.min().item(),
            'qe_max': qe.max().item(),
            'qe_mean': qe.mean().item(),
            'clamp_hits': clamp_hits,
        }
    return qe, diagnostics
return qe

"""
if __name__ == "__main__":
    initial_k2 = torch.tensor([[ -2.0]])
    initial_qe = torch.tensor([[ 1.25]])
    initial_kf = torch.tensor([[ 0.673]])
    initial_n = torch.tensor([[ 0.23]])
    input_size = 3
    hidden_size = 16
    output_size = 1
    pinn3D_model = PINN3DModel(input_size = input_size,
                               hidden_size = hidden_size,
                               output_size = output_size,
                               initial_k2= initial_k2,
                               initial_qe = initial_qe,
                               initial_kf = initial_kf,
                               initial_n = initial_n)
    print(pinn3D_model.state_dict())
```


A.3 src/pinn3D/train.py

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import pandas as pd
import numpy as np
from pathlib import Path
# Prefer the qualified package import: src/pinn1D also has a bare-named
# data.py, and if that one gets imported first into sys.modules under the
# name "data" (e.g. from notebook cells that mix both packages), a plain
# `from data import ...` here would silently resolve to the wrong module.
try:
    from pinn3D.data import CombinedData, load_data
    from pinn3D.models import PINN3DModel, predict_qe
except ImportError:
    from data import CombinedData, load_data
    from models import PINN3DModel, predict_qe
import warnings
warnings.filterwarnings('ignore')
import time

PROJECT_ROOT = Path(__file__).resolve().parents[2]
RESULTS_DIR = PROJECT_ROOT / 'results'

EPOCHS = 8000
# PSO loss needs torch.autograd.grad(create_graph=True) through LayerNorm/Tanh
# (double backward); PyTorch's MPS backend produces NaN gradients for some
# inputs on this path, so training must run on CPU.
DEVICE = torch.device('cpu')

# Freundlich exponent n is applied directly to raw concentration (up to ~65),
# so  $n > -1$  makes  $q_{e\_real}$  blow up ( $65^{**2} \sim 4200$ ,  $65^{**3} \sim 275000$ ). Real
# adsorption data tops out at ~2.5, which matches  $n$  in (0,1) (the usual
# sub-linear Freundlich regime) -- so  $n$  is bounded there instead of left
# unbounded via softplus.
N_MIN = 0.05
N_MAX = 1.0
# Without a floor,  $k_2$  can be driven to ~0 during training, which trivially
# zeroes the PSO residual ( $k_2 * (q_e - q_t)^{**2}$  term vanishes) without the model
# ever learning real kinetics -- this also lets  $k_f/n$  drift unchecked since
# the ODE constraint on them disappears. The floor keeps that constraint live.
K2_FLOOR = 1e-3

# --- Fixed manual loss weights (loss-spike fix, see PINN3D_knowledge_transfer.md
# Section 4) ---
# Diagnosis: total_loss summed the four terms unweighted. PSO Loss reaches a
# steady-state floor (~4.3e-5) that is 10-180x LARGER than the other three
# terms' floors, while also being the flattest (lowest epoch-to-epoch
# variance) of the four. Because Adam's per-parameter adaptive scaling
# normalizes by each parameter's *combined* gradient variance (all four terms
# are summed before backward()), that persistently-large-but-steady PSO
# gradient was getting an outsized effective step relative to the noisier
# Data/Initial/ISO gradients, which share the same hidden representation --
# producing the observed ~10-epoch cycles where Data/Initial/ISO loss rise
# and fall together while PSO stays flat. This was confirmed to NOT be caused
# by predict_qe's fixed-point solve ( $K_2/K_f/N$  moved smoothly through a spike,
# Clamp_hits stayed 0).
#
# Weights below bring all four terms to a comparable order of magnitude so no
# single term's gradient dominates the shared representation. Derived from
# the median of each raw loss column over the last 2000 epochs (steady-state
# region, post burn-in) of results/pinn3d_training_results.csv from the
# pre-fix run:
# Data Loss median ~2.35e-7, PSO Loss median ~4.32e-5,
# Initial Loss median ~1.20e-6, ISO Loss median ~5.18e-6
# weight_i = (PSO median) / (term_i median), i.e. PSO_LOSS_WEIGHT=1 is the
# reference scale and the other three are scaled up to match it. Re-derive
# these (same formula, against a fresh training_results.csv) if the loss
```

```

# floors shift meaningfully after retraining with these weights in place --
# they are a starting point, not a permanent constant.
DATA_LOSS_WEIGHT = 180.0
PSO_LOSS_WEIGHT = 1.0
INITIAL_LOSS_WEIGHT = 35.0
ISO_LOSS_WEIGHT = 8.0

def transform_k2(raw_k2):
    return F.softplus(raw_k2) + K2_FLOOR

def transform_n(raw_n):
    return N_MIN + (N_MAX - N_MIN) * torch.sigmoid(raw_n)

def training_pinn3d(data: CombinedData,
                    model: PINN3DModel,
                    optimizer: torch.optim.Optimizer,
                    loss_fn: torch.nn.Module,
                    epochs: int = EPOCHS,
                    device = DEVICE):
    result_dict = {
        'Data Loss': [],
        'PSO Loss': [],
        'Initial Loss': [],
        'ISO Loss': [],
        'Weighted Total Loss': [],
        # --- diagnostics for the loss-spike investigation ---
        'K2': [],
        'Kf': [],
        'N': [],
        'Qe_min': [],
        'Qe_max': [],
        'Qe_mean': [],
        'Clamp_hits': [],
    }
    model.train()
    model.to(device)
    # get the input, output and collocation data
    co = data.collocated_input[:, 1:2]
    co = torch.tensor(co, dtype = torch.float32, device = device)
    dosage = data.collocated_input[:, 2:3]
    dosage = torch.tensor(dosage, dtype = torch.float32, device = device)
    # pass the data into the allocated device
    data.pass_to_device([data.scaled_input_tensor, data.scaled_output_tensor, data.collocated_scaled_input_tensor,
                        data.initial_scaled_datapoints_tensor,
                        data.max_time_tensor])
    # The model is trained to output STANDARDIZED adsorption (matched against
    # scaled_output_tensor in data_loss). qe_real from predict_qe() and the
    # initial/equilibrium physics targets are all in REAL mg/g units, so every
    # model output touching them has to be unscaled first, or the physics
    # losses are enforced against the wrong absolute scale and offset.
    output_mean = torch.tensor(data.output_mean, dtype = torch.float32, device = device)
    output_std = torch.tensor(data.output_std, dtype = torch.float32, device = device)

    def unscale_output(scaled_output):
        return scaled_output * output_std + output_mean

    for epoch in range(epochs):
        optimizer.zero_grad()
        # pass the input to the model
        pred_output = model(data.scaled_input_tensor)
        # get the loss
        data_loss = loss_fn(pred_output, data.scaled_output_tensor.view(-1, 1))
        result_dict['Data Loss'].append(data_loss.item())
        # pso loss
        collocation_pred_adsorption = model(data.collocated_scaled_input_tensor)
        # scale the learnable parameters
        positive_k2 = transform_k2(model.k2)
        positive_kf = F.softplus(model.kf)
        positive_n = transform_n(model.n)
        qe_real, qe_diag = predict_qe(co = co,

```

```

        dosage = dosage,
        kf = positive_kf,
        n = positive_n,
        return_diagnostics = True)

qe_real.to(device)
# log the physical-parameter / fixed-point diagnostics every epoch --
# this is what we're using to confirm the loss-spike hypothesis:
# a jump in Qe_min/Qe_max/Clamp_hits should line up with the spikes.
result_dict['K2'].append(positive_k2.item())
result_dict['Kf'].append(positive_kf.item())
result_dict['N'].append(positive_n.item())
result_dict['Qe_min'].append(qe_diag['qe_min'])
result_dict['Qe_max'].append(qe_diag['qe_max'])
result_dict['Qe_mean'].append(qe_diag['qe_mean'])
result_dict['Clamp_hits'].append(qe_diag['clamp_hits'])
dq_dt_n = torch.autograd.grad(
    outputs = collocation_pred_adsorption.sum(),
    inputs = data.collocated_scaled_input_tensor,
    create_graph = True
)[0][:, 0:1] # dq/dt, dq/dc, dq/ddosage
# dq/dt = k2*(qe-qt)**2
# dq_dt_n is d(scaled_output)/d(scaled_time). Chain rule to get
# d(real_output)/d(real_time): multiply by d(scaled_time)/d(real_time)
# = 1/t_std, AND by d(real_output)/d(scaled_output) = output_std.
dq_dt_real = dq_dt_n * output_std * (1/data.input_stats['Time']['std'])
collocation_pred_adsorption_real = unscale_output(collocation_pred_adsorption)
pso_loss = torch.mean((dq_dt_real - (positive_k2 * (qe_real - collocation_pred_adsorption_real)**2))**2)
result_dict['PSO Loss'].append(pso_loss.item())
# initial loss (real adsorption must be 0 at t=0, not the scaled output)
initial_adsorption_pred = model(data.initial_scaled_datapoints_tensor)
initial_adsorption_pred_real = unscale_output(initial_adsorption_pred)
initial_loss = torch.mean(initial_adsorption_pred_real**2)
result_dict['Initial Loss'].append(initial_loss.item())
# isotherm loss (Using Freundlich equation)
# Ce = Co - qe/vm
# qe = k_f * Ce**n
# iso loss --> # qe_real: the max adsorption, collocation_pred_adsorption:
max_adsorption_pred = model(data.max_time_tensor)
max_adsorption_pred_real = unscale_output(max_adsorption_pred)
iso_loss = torch.mean((qe_real - max_adsorption_pred_real)**2)
result_dict['ISO Loss'].append(iso_loss.item())
# weighted sum (see DATA_LOSS_WEIGHT et al. above) -- this is what
# actually gets backpropagated; the four raw losses logged above stay
# unweighted so they're directly comparable to the pre-fix run.
total_loss = (DATA_LOSS_WEIGHT * data_loss
              + PSO_LOSS_WEIGHT * pso_loss
              + INITIAL_LOSS_WEIGHT * initial_loss
              + ISO_LOSS_WEIGHT * iso_loss)
result_dict['Weighted Total Loss'].append(total_loss.item())
total_loss.backward()
optimizer.step()

# print
if (epoch+1)%100 == 0:
    flag = f" | !! clamp_hits={qe_diag['clamp_hits']} |!" if qe_diag['clamp_hits'] > 0 else ""
    print(f"Epoch:{epoch+1}/{epochs} | Data Loss: {data_loss.item():.3f} | PSO Loss: {pso_loss.item():.3f} | Init{flag}")
# make the result a dictionary
print('-'*120)
print(f"Learnable Parameters")
print('-'*120)
print(f"Rate Constant (K2):{transform_k2(model.k2).item()}")
print(f"Freundlich Constant (Kf):{F.softplus(model.kf)}")
print(f"N:{transform_n(model.n)}")
print(f"Total Loss:")
result_df = pd.DataFrame(result_dict)
result_df.insert(0, 'Epoch', range(1, len(result_df) + 1))
result_df['Total Loss'] = result_df[['Data Loss', 'PSO Loss', 'Initial Loss', 'ISO Loss']].sum(axis=1)
return result_df

def save_training_results(result_df:pd.DataFrame, path=RESULTS_DIR / 'pinn3d_training_results.csv'):

```

```

path = Path(path)
path.parent.mkdir(parents=True, exist_ok=True)
result_df.to_csv(path, index=False)
return path

def save_parameters(model:PINN3DModel, result_df:pd.DataFrame, path=RESULTS_DIR / 'pinn3d_parameters.md'):
    """Write the learned physical parameters and final loss values to a markdown file."""
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    k2 = transform_k2(model.k2).item()
    kf = F.softplus(model.kf).item()
    n = transform_n(model.n).item()
    final_row = result_df.iloc[-1]
    lines = [
        "# PINN3D Learned Parameters",
        "",
        f"- Trained: {time.strftime('%Y-%m-%d %H:%M:%S')}",
        f"- Epochs: {len(result_df)}",
        "",
        "## Physical Parameters",
        "",
        f"- Rate Constant (K2): {k2:.6f}",
        f"- Freundlich Constant (Kf): {kf:.6f}",
        f"- Freundlich Exponent (N): {n:.6f}",
        "",
        "## Final Losses",
        "",
        f"- Data Loss: {final_row['Data Loss']:.6f}",
        f"- PSO Loss: {final_row['PSO Loss']:.6f}",
        f"- Initial Loss: {final_row['Initial Loss']:.6f}",
        f"- ISO Loss: {final_row['ISO Loss']:.6f}",
        f"- Total Loss: {final_row['Total Loss']:.6f}",
        ""
    ]
    path.write_text("\n".join(lines))
    return path

if __name__ == "__main__":
    torch.manual_seed(42)
    np.random.seed(42)
    data_path = "/Users/tinonturjamajumder/Catkin_Biofilm_Adsorption/data of biofilm(TINON BHAI).xlsx"
    data = load_data(data_path)
    INPUT_SIZE = 3
    HIDDEN_SIZE = 32
    OUTPUT_SIZE = 1
    INITIAL_K2 = torch.tensor([-3.0], dtype = torch.float32)
    INITIAL_KF = torch.tensor([-1.0], dtype = torch.float32)
    # raw input to transform_n (bounded sigmoid, not softplus) -> n starts at ~0.52
    INITIAL_N = torch.tensor([0.0], dtype = torch.float32)
    model = PINN3DModel(input_size = INPUT_SIZE,
                        hidden_size = HIDDEN_SIZE,
                        output_size = OUTPUT_SIZE,
                        initial_k2 = INITIAL_K2,
                        initial_kf = INITIAL_KF,
                        initial_n = INITIAL_N)
    OPTIMIZER = torch.optim.Adam(model.parameters(), lr = 0.005)
    LOSS_FN = torch.nn.MSELoss()
    start_time = time.time()
    print("="* 120)
    print("Training Start")
    print("="* 120)
    result_df = training_pinn3d(data = data,
                                model = model,
                                optimizer = OPTIMIZER,
                                loss_fn = LOSS_FN,
                                )
    print("="* 120)
    print("Training End")
    print("="* 120)
    end_time = time.time()

```

```
total_time = end_time - start_time
print(f"Time recorded to train: {total_time}")
results_path = save_training_results(result_df)
parameters_path = save_parameters(model, result_df)
print(f"Saved training loss history to: {results_path}")
print(f"Saved learned parameters to: {parameters_path}")
```